Session 04 - Saving Employees and Documents to Files - Tests

Python Contoso Vault - Session 04 of 07

Part 1 - Core part A: vault.py

Goal

Make `vault.py` remember documents after the user quits. Save them to a JSON file each time a document is created, and load them again when the program starts. Keep the bodies encrypted in the file.

Run `python vault.py` in the terminal. Type each input line and press Enter after it. Delete `documents.json` before each test that starts in an empty folder. To try one function, open the Python shell in the same folder and run `from vault import load_documents, save_documents`.

Provided

Copy your own `caesar.py` and `vault.py` from your Session 03 folder into this folder. `vault.py` already defines `create_document(documents, title, body, key) -> dict` and `check_key(document, key) -> bool`, and runs the four-line menu.

Instructions

Amend the copied `vault.py`. Keep `create_document`, `check_key` and the menu as they are, and add these steps to the same file.

1. Import the `json` module.
2. Define a function `load_documents` that takes the path of a JSON file as a string.
   2.1. Return the documents stored in the JSON file as a list of dictionaries.
   2.2. Return an empty list if the file does not exist.
3. Define a function `save_documents` that takes the documents as a list and the path of a JSON file as a string.
   3.1. Write the documents to the JSON file, in place of what the file held before.
4. When the file runs as a program:
   4.1. Load the documents from `documents.json` at the start, in place of the empty list.
   4.2. On `1`, save the documents to `documents.json` after the new document is created and before `Saved document N.` prints.
   4.3. Keep every step of the Session 03 program that the steps above do not change.

Constraints

- Every constraint in the Session 03 `vault.py` test plan still holds.
- `documents.json` sits in the folder the program runs from.
- The file holds one JSON list of the documents as `create_document` returns them.
- New IDs carry on from the loaded documents. With two documents loaded, the next document has ID 3.
- The program writes the file only when a document is created. Listing, reading and quitting write nothing.
- `load_documents` and `save_documents` neither print nor read input.
- Importing the file reads no input, prints nothing and writes no file.

Design

The following rules say how to write the code. Follow them.

- Every Design point in the Session 03 `vault.py` test plan still applies.
- Save the documents before `Saved document N.` prints.
- Give every variable and function a type hint.

Input validation

Required for every key and ID, as in Session 03. Not required for the file. Assume `documents.json` is missing or was written by `save_documents`.

Test plan

Function tests

`document_1` is `{"id": 1, "title": "Payroll", "body": "Vdodulhv jr rxw rq 28 Qryhpehu.", "check": "FRQWRVR-YDXOW"}`. Likewise, `document_2` is `{"id": 2, "title": "Kandy rent", "body": "Vlyoj zqqtnp cpye td Cd 150,000.", "check": "NZYEZDZ-GLFWE"}`. Here, `path` names a file in an empty folder.

Test	Call	Expected output	Actual output	Pass
1	`load_documents(path)` with no file at `path`	`[]`		
2	`save_documents([document_1], path)`, then `load_documents(path)`	`[document_1]`		
3	`save_documents([document_1], path)`, then read the file as text	Holds `Vdodulhv jr rxw rq 28 Qryhpehu.` and never `Salaries`		
4	`save_documents([document_1], path)`, then `save_documents([document_1, document_2], path)`, then `load_documents(path)`	`[document_1, document_2]`		
5	`save_documents([], path)`, then `load_documents(path)`	`[]`		
6	`save_documents([document_1], path)`	Returns `None`		

Program tests

Each test runs `python vault.py` in an empty folder, once or twice in a row. "Create Payroll" stands for the four input lines `"1"`, `"Payroll"`, `"Salaries go out on 28 November."`, `"3"`. "Create Kandy rent" stands for `"1"`, `"Kandy rent"`, `"Kandy office rent is Rs 150,000."`, `"11"`. Every run prints the four menu lines and the prompt `"Choice: "` before each choice, and ends with the `Goodbye.` line. For the last run, the expected output lists the output lines that are not menu lines, prompts or `Goodbye.`, such as `Saved document 1.` Then it gives the content of `documents.json` after the last run.

Test	First run input lines	Second run input lines	Expected output	Actual output	Pass
7	`4`	None	Only the menu and `Goodbye.` `documents.json`: none		
8	`2`, `3`, `1`, `4`	None	`No documents.`, `No such document.` `documents.json`: none		
9	Create Payroll, `4`	None	`Saved document 1.` `documents.json`: `[document_1]`		
10	Create Payroll, `4`	`2`, `4`	`1. Payroll` `documents.json`: `[document_1]`		
11	Create Payroll, `4`	`3`, `1`, `3`, `4`	`Salaries go out on 28 November.` `documents.json`: `[document_1]`		
12	Create Payroll, `4`	Create Kandy rent, `2`, `4`	`Saved document 2.`, `1. Payroll`, `2. Kandy rent` `documents.json`: `[document_1, document_2]`		
13	Create Payroll, `4`	`3`, `1`, `5`, `6`, `7`, `2`, `4`	`Wrong key.` three times, `Too many attempts.`, `1. Payroll` `documents.json`: `[document_1]`		
14	`import vault`	None	Reads no input and prints nothing `documents.json`: none		
15	Each Session 03 program test, 9 to 20	None	The Session 03 expected output lines		

In every program test, the program ends after `Goodbye.` and shows no Python error message.

Done criteria

- [ ] Every test plan row gives exactly the expected result.
- [ ] The script runs from the terminal with `python vault.py`.

Part 2 - Core part B: import_employees.py

Goal

Contoso HR sends a spreadsheet export of new employees, and some rows have problems. Clean the good rows and add them to `employees.json`, the employee file of Contoso's document vault. List every rejected row with one reason, so HR can fix it. Make sure a second run adds nobody twice.

Open the Python shell in a folder that holds a copy of `employees_export.csv`, run `from import_employees import import_employees`, then make each call in the test plan. Delete `employees.json` and `rejected.csv` before each test that starts with no `employees.json`.

Provided

`employees_export.csv` arrives in this folder when you sync your fork. It holds a header and 12 employee rows. The file starts with a UTF-8 byte-order mark. Row 1 is the header, so the first employee is row 2. Here are the first four rows of the file:

```
employee_id,name,email,department,start_date
E1001,Nimali Perera,nimali.perera@contoso.lk,Finance,2026-11-02
E1002,  Kasun Silva ,kasun.silva@contoso.lk, sales ,2026-11-03
E1003,Ayesha Fernando,,Human Resources,2026-11-03
```

Instructions

1. Import the `csv` and `json` modules.
2. Define a function `import_employees` that takes the path of the CSV file as a string and the path of the JSON employee file as a string.
   2.1. Load the stored employees from the JSON employee file into the employees list, a list of dictionaries. Start the employees list empty if the file does not exist.
   2.2. Start the rejected list as an empty list.
   2.3. Read the CSV rows in order. For each row:
      2.3.1. Remove the spaces at the start and end of every field.
      2.3.2. If the email is empty, add the row number as an integer and the reason `MISSING_EMAIL` to the rejected list, as a tuple, and go on to the next row.
      2.3.3. If an employee in the employees list already has the row's employee ID, add the row number as an integer and the reason `DUPLICATE_ID` to the rejected list, as a tuple, and go on to the next row.
      2.3.4. Otherwise, change the row's department to title case and add the row to the employees list, as a dictionary.
   2.4. Save the employees list to the JSON employee file.
   2.5. Write `rejected.csv` in the same folder as the JSON employee file. Write the header `row,reason` first, then one line per tuple in the rejected list.
   2.6. Return the number of rows added to the employees list and the number of rejected rows, as a tuple of two integers.

Constraints

- The JSON employee file holds one list of employees. Each employee is a dictionary with exactly the keys `employee_id`, `name`, `email`, `department` and `start_date`. Every value is a string. A file with one employee holds:

```
[
  {
    "employee_id": "E1002",
    "name": "Kasun Silva",
    "email": "kasun.silva@contoso.lk",
    "department": "Sales",
    "start_date": "2026-11-03"
  }
]
```

- The first key is `employee_id`. The byte-order mark never appears in a key or a value.
- Stored employees keep their order. Accepted rows follow them in CSV order.
- Only rows in the employees list count as duplicates. When a row is rejected for a missing email, a later row with the same ID is still added. Row 4 (`E1003`, no email) is rejected, and row 8 (`E1003`) is added.
- `rejected.csv` sits in the same folder as the employee file. Its header is `row,reason`. Each line holds the spreadsheet row number and the reason code, in CSV order. It holds only the header when no row is rejected. A new run replaces it.
- A second run on the same CSV adds no row, and the employee file keeps the same employees.
- The function neither prints nor reads input.
- Importing the file reads no input, prints nothing and writes no file.

Design

The following rules say how to write the code. Follow them.

- Give every variable and function a type hint.

Input validation

Only the email and the employee ID, as the instructions set out. Assume the CSV file exists, every row has an employee ID, every date is in the form `2026-11-22`, and the employee file is missing or holds a list written by this function.

Test plan

Each test runs in an empty folder that holds a copy of `employees_export.csv`. The employee file is `employees.json` in that folder. The expected output gives the return value, then the `rejected.csv` lines after the header, then `employees.json`.

Test	Call	Expected output	Actual output	Pass
1	`import_employees( "employees_export.csv", "employees.json")` with no `employees.json`	Returns `(7, 5)` `rejected.csv`: `4,MISSING_EMAIL`, `6,DUPLICATE_ID`, `7,MISSING_EMAIL`, `10,DUPLICATE_ID`, `12,MISSING_EMAIL` `employees.json`: 7 employees with IDs `E1001`, `E1002`, `E1004`, `E1003`, `E1006`, `E1007`, `E1008`, in that order		
2	Same as test 1	Returns `(7, 5)` `employees.json`: `E1002` is `{"employee_id": "E1002", "name": "Kasun Silva", "email": "kasun.silva@contoso.lk", "department": "Sales", "start_date": "2026-11-03"}`		
3	Same as test 1	Returns `(7, 5)` `employees.json`: `E1004` has department `Customer Service`, `E1003` has email `ayesha.fernando@contoso.lk` and department `Human Resources`, and `E1008` has name `Kavindi Herath` and email `kavindi.h@contoso.lk`		
4	Test 1, then the same call again	Returns `(0, 12)` `rejected.csv`: `2,DUPLICATE_ID`, `3,DUPLICATE_ID`, `4,MISSING_EMAIL`, `5,DUPLICATE_ID`, `6,DUPLICATE_ID`, `7,MISSING_EMAIL`, `8,DUPLICATE_ID`, `9,DUPLICATE_ID`, `10,DUPLICATE_ID`, `11,DUPLICATE_ID`, `12,MISSING_EMAIL`, `13,DUPLICATE_ID` `employees.json`: same as after test 1		
5	Same as test 1, with `employees.json` already holding the `E1007` employee from test 1	Returns `(6, 6)` `rejected.csv`: `4,MISSING_EMAIL`, `6,DUPLICATE_ID`, `7,MISSING_EMAIL`, `10,DUPLICATE_ID`, `11,DUPLICATE_ID`, `12,MISSING_EMAIL` `employees.json`: 7 employees with IDs `E1007`, `E1001`, `E1002`, `E1004`, `E1003`, `E1006`, `E1008`, in that order		
6	Same as test 1, on a CSV with only the header row and no byte-order mark	Returns `(0, 0)` `rejected.csv`: no lines `employees.json`: `[]`		
7	`import import_employees`	Reads no input, prints nothing and writes no file		

Done criteria

☐ Every test plan row gives exactly the expected result.

Part 3 - Challenge: clean_rows.py

Goal

Read Contoso HR's spreadsheet export one row at a time. Give back each cleaned row, with its spreadsheet row number, only when a program asks for the next row. That way, an export of any size never fills the computer's memory.

Open the Python shell in the `04-saving-employees-and-documents-to-files` folder and run `from clean_rows import clean_rows`, then make each call in the test plan.

Provided

You use the same `employees_export.csv` as in Core part B. It starts with a UTF-8 byte-order mark, and row 1 is the header.

Instructions

1. Import the `csv` module, and `Iterator` from the `collections.abc` module.
2. Define a generator function `clean_rows` that takes the path of the CSV file as a string.
    2.1. Read the CSV rows in order. For each row:
        2.1.1. Remove the spaces at the start and end of every field.
        2.1.2. Yield the spreadsheet row number as an integer and the cleaned row as a dictionary, as a tuple.

Constraints

- Calling `clean_rows` reads nothing from the file. The file opens at the first `next()`.
- The first row it yields has row number 2, because row 1 is the header. Each later row number is one more than the row number before it.
- The keys come from the header. The first key is `employee_id`, and the byte-order mark never appears in a key or value.
- It yields every row in the file, including rows with an empty field. It changes no value other than removing the spaces.
- The function neither prints nor reads input.
- Importing the file reads no input, prints nothing and writes no file.

Design

The following rules say how to write the code. Follow them.

- Build `clean_rows` with `yield`.
- Give every variable and function a type hint.

Input validation

Not required. Assume the CSV file has a header row.

Test plan

Each test runs in a folder that holds a copy of `employees_export.csv`. Here, `rows` stands for `clean_rows("employees_export.csv")`.

Test	Call	Expected output	Actual output	Pass
1	`next(rows)`	`(2, {"employee_id": "E1001", "name": "Nimali Perera", "email": "nimali.perera@contoso.lk", "department": "Finance", "start_date": "2026-11-02"})`		
2	`next(rows)` a second time, on the same `rows`	`(3, {"employee_id": "E1002", "name": "Kasun Silva", "email": "kasun.silva@contoso.lk", "department": "sales", "start_date": "2026-11-03"})`		
3	`list(rows)`	12 tuples, with row numbers 2 to 13 in order		
4	The third tuple of `list(rows)`	`(4, {"employee_id": "E1003", "name": "Ayesha Fernando", "email": "", "department": "Human Resources", "start_date": "2026-11-03"})`		
5	The last tuple of `list(rows)`	`(13, {"employee_id": "E1008", "name": "Kavindi Herath", "email": "kavindi.h@contoso.lk", "department": "Sales", "start_date": "2026-11-23"})`		
6	`next(rows)` after the 12th row	Raises `StopIteration`		
7	`clean_rows("missing.csv")`	Raises nothing		
8	`next(clean_rows("missing.csv"))`	Raises `FileNotFoundError`		
9	`import clean_rows`	Reads no input and prints nothing		

Done criteria

- ☐ Every test plan row gives exactly the expected result.

Marking

This take-home is marked out of 100, with a pass mark of 60.

Part	Tests	Design	Total
Core	64	16	80
Challenge	16	4	20
Total	**80**	**20**	**100**

Guidelines

- Test marks show that the program works. The instructor's tests run it on every row of the test plan, plus further cases.
- Design marks show that the code is built well. The instructor reads the code against each Design rule.
- A pass needs 60 marks from the Core alone. Challenge marks count towards the total once the Core reaches 60.
- Marking uses the last commit pushed by 11:59 PM on the day before the next session.
- The submission is one GitHub link to the fork, posted on the portal.